

# Programming for AI

## AI-2001

Sequences: Lists and Tuples

---

National University of Computer & Emerging Sciences – NUCES – Karachi



# National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2001

Programming for AI

## Contents

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| 1. Introduction .....                                                           | 3  |
| 2. Lists.....                                                                   | 3  |
| 2.1. Creating a List .....                                                      | 3  |
| 2.1. Accessing Elements of a List .....                                         | 3  |
| 2.2. Determining List's Length .....                                            | 4  |
| 2.3. Accessing Elements from the End of the List with Negative Indices .....    | 4  |
| 2.4. Indices Must Be Integers or Integer Expressions .....                      | 4  |
| 2.5. Lists Are Mutable .....                                                    | 4  |
| 2.6. Some Sequences Are Immutable .....                                         | 5  |
| 2.7. Attempting to Access a Nonexistent Element .....                           | 5  |
| 2.8. Using List Elements in Expressions .....                                   | 5  |
| 2.9. Appending to a List with += .....                                          | 5  |
| 2.9. Concatenating Lists with + .....                                           | 6  |
| 2.10. Using for and range to Access List Indices and Values .....               | 6  |
| 2.11. Comparison Operators.....                                                 | 7  |
| 3. Tuples.....                                                                  | 7  |
| 3.1. Creating Tuples.....                                                       | 7  |
| 3.2. Accessing Tuple Elements .....                                             | 8  |
| 3.3. Adding Items to a String or Tuple .....                                    | 8  |
| 3.4. Appending Tuples to Lists .....                                            | 9  |
| 3.5. Tuples May Contain Mutable Objects .....                                   | 9  |
| 4. Unpacking Sequences .....                                                    | 10 |
| 4.1. Swapping Values Via Packing and Unpacking .....                            | 10 |
| 4.2. Accessing Indices and Values Safely with Built-in Function enumerate ..... | 11 |
| 5. Sequence Slicing .....                                                       | 11 |
| 5.1. Specifying a Slice with Starting and Ending Indices.....                   | 11 |
| 5.2. Specifying a Slice with Starting and Ending Indices .....                  | 12 |
| 5.3. Specifying a Slice with Only a Starting Index .....                        | 12 |
| 5.4. Specifying a Slice with No Indices.....                                    | 12 |

|                                                         |    |
|---------------------------------------------------------|----|
| 5.5 Slicing with Steps .....                            | 12 |
| 5.6 Slicing with Negative Indices and Steps.....        | 13 |
| 5.7 Modifying Lists Via Slices .....                    | 13 |
| 6. del Statement .....                                  | 14 |
| 6.1 Deleting the Element at a Specific List Index ..... | 14 |
| 6.2 Deleting a Slice from a List.....                   | 14 |
| 6.3 Deleting a Slice Representing the Entire List ..... | 14 |
| 6.4 Deleting a Variable from the Current Session.....   | 14 |

Talha Shahid

# 1. Introduction

Collections, such as a contact list or a shopping list, are data structures that group related items together. Python offers built-in collections that make storing and accessing data simple and efficient. In this manual, lists and tuples are explored in detail, demonstrating their common manipulations. Lists are modifiable and can dynamically change in size, while tuples are immutable. Both can contain items of various types. The manual also covers one- and two-dimensional lists. The previous chapter discussed random-number generation and die roll simulations. This manual concludes with a section on data visualization using Seaborn and Matplotlib, where static bar charts display die frequencies. The next chapter will build on this by introducing animated visualizations that dynamically update with more die rolls, illustrating the law of large numbers.

## 2. Lists

### 2.1. Creating a List

Lists usually store homogeneous data, meaning values of the same data type. For example, consider the list `c`, which holds five integer elements:

```
[1] c = [-45, 6, 0, 72, 1543]
```

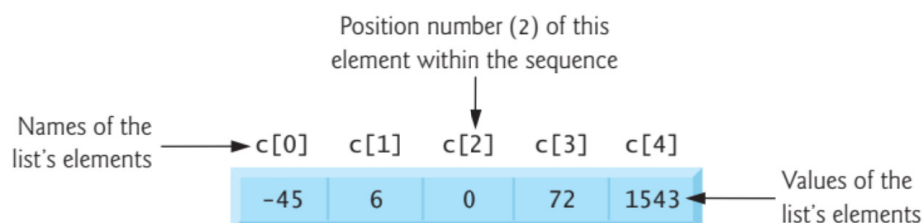
```
[2] c
[-45, 6, 0, 72, 1543]
```

They can also hold diverse types of data, meaning data of various kinds. For instance, the following list includes a student's first name (a **string**), last name (a **string**), grade point average (a **float**), and graduation year (an **integer**):

```
['Mary', 'Smith', 3.57, 2022]
```

### 2.1. Accessing Elements of a List

To access an element in a list, you write the list's name followed by the element's index (its position in the list) inside square brackets (`[]`, called the subscription operator). The diagram below illustrates the list `c` with its elements labelled accordingly.



The index of the first element in a list is `0`. Therefore, in a list `c` with five elements, the first element is referred to as `c[0]`, and the last element is `c[4]`.

```
[3] c[0]
-45
```

```
[4] c[4]
```

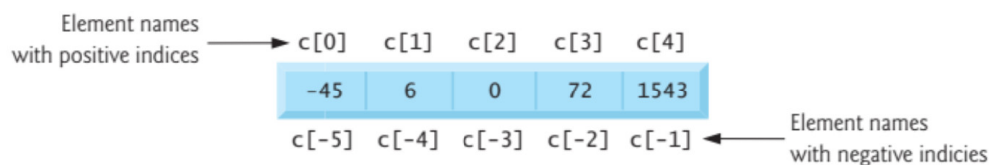
## 2.2. Determining List's Length

You can determine the length of a list by using the built-in **len** function.

```
[5] | len(c)
      5
```

## 2.3. Accessing Elements from the End of the List with Negative Indices

You can also access elements in a list from the end by using negative indices.



Thus, the last element of list **c** (**c[4]**) can be accessed using **c[-1]**, while the first element can be retrieved with **c[-5]**.

```
[6] | c[-1]
      1543
[7] | c[-5]
      -45
```

## 2.4. Indices Must Be Integers or Integer Expressions

An index must be either an integer, an integer expression, or a slice (which will be covered shortly).

```
[8] | a = 1
[9] | b = 2
[10] | c[a + b]
      72
```

Using a non-integer index value causes a **TypeError**.

## 2.5. Lists Are Mutable

Lists are mutable—their elements can be modified:

```
[11] | c[4] = 17
[12] | c
      [-45, 6, 0, 72, 17]
```

You'll soon see that you also can insert and delete elements, changing the list's length.

## 2.6. Some Sequences Are Immutable

In Python, strings and tuples are immutable, meaning they cannot be changed. While you can access individual characters in a string, trying to modify any of them will result in a **TypeError**.

```
[13] s = 'hello'
```

```
[14] s[0]
```

```
[15] s[0] = 'H'
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[14], line 1  
----> 1 s[0] = 'H'  
  
TypeError: 'str' object does not support item assignment
```

## 2.7. Attempting to Access a Nonexistent Element

Using an out-of-range list, tuple or string index causes an **IndexError**:

```
[16] c[100]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[15], line 1  
----> 1 c[100]  
  
IndexError: list index out of range
```

## 2.8. Using List Elements in Expressions

List elements may be used as variables in expressions:

```
[17] c[0] + c[1] + c[2]  
-39
```

## 2.9. Appending to a List with +=

We'll begin with an empty list `[]` and use a **for** loop along with the **+=** operator to add the values **1** through **5** to it. The list will expand automatically to hold each new item.

```
[18] a_list = []
```

```
[19] for number in range(1, 6):  
      a_list += [number]
```

```
[20] a_list
      [1, 2, 3, 4, 5]
```

When using `+=` with a list as the left operand, the right operand must be an iterable; otherwise, a **TypeError** will be raised. In snippet [19], the square brackets around number form a one-item list, which is added to `a_list`. If the right operand has multiple elements, `+=` appends all of them. For example, the following adds each character of `'Python'` to the `letters` list.

```
[21] letters = []
```

```
[22] letters += 'Python'
```

```
[23] letters
      ['P', 'y', 't', 'h', 'o', 'n']
```

When the right operand of `+=` is a tuple, its elements are added to the list as well.

## 2.9. Concatenating Lists with +

You can combine two lists, tuples, or strings using the `+` operator. This creates a new sequence of the same type, containing the elements from the left operand followed by those from the right operand, without altering the original sequences.

```
[24] list1 = [10, 20, 30]
```

```
[25] list2 = [40, 50]
```

```
[26] concatenated_list = list1 + list2
```

```
[27] concatenated_list
      [10, 20, 30, 40, 50]
```

A **TypeError** will occur if the operands of the `+` operator are different types of sequences—for instance, attempting to concatenate a list with a tuple will result in an error.

## 2.10. Using for and range to Access List Indices and Values

List elements also can be accessed via their indices and the subscription operator (`[]`):

```
[28] for i in range(len(concatenated_list)):
      print(f'{i}: {concatenated_list[i]}')
      0: 10
      1: 20
      2: 30
```

---

```
3: 40
4: 50
```

The function call `range(len(concatenated_list))` generates a sequence of integers that represent the indices of `concatenated_list` (in this case, from `0` to `4`). When using this approach in a loop, it's important to make sure the indices stay within the valid range.

## 2.11. Comparison Operators

You can compare entire lists element-by-element using comparison operators:

```
[29] a = [1, 2, 3]
```

```
[30] b = [1, 2, 3]
```

```
[31] c = [1, 2, 3, 4]
```

```
[32] a == b # True: corresponding elements in both are equal
True
```

```
[33] a == c # False: a and c have different elements and lengths
False
```

```
[34] a < c # True: a has fewer elements than c
True
```

```
[35] c >= b # True: elements 0-2 are equal but c has more elements
True
```

## 3. Tuples

Tuples are immutable and usually hold heterogeneous data, though they can also contain homogeneous data. The length of a tuple, which is the number of its elements, remains constant throughout the program's execution.

### 3.1. Creating Tuples

To create an empty tuple, use empty parentheses:

```
[1] student_tuple = ()
```

```
[2] student_tuple
()
```

```
[3] len(student_tuple)
0
```

```
[4] student_tuple = 'John', 'Green', 3.3
```

```
[5] student_tuple
student_tuple
```

```
[6] len(student_tuple)
3
```

When you print a tuple, Python always shows its contents within parentheses. You can also optionally enclose a tuple's comma-separated values in parentheses.

```
[7] another_student_tuple = ('Mary', 'Red', 3.3)
```

```
[8] another_student_tuple  
('Mary', 'Red', 3.3)
```

The following code creates a one-element tuple:

```
[9] a_singleton_tuple = ('red',) # note the comma
```

```
[10] a_singleton_tuple  
('red',)
```

The comma (,) after the string 'red' designates **a\_singleton\_tuple** as a tuple—the parentheses are not required. Without the comma, the parentheses would be unnecessary, and **a\_singleton\_tuple** would just refer to the string 'red' instead of being a tuple.

## 3.2. Accessing Tuple Elements

The elements of a tuple, while related, often come in different types. Typically, you don't iterate over them; instead, you access each element individually. Just like list indices, tuple indices start at 0. The following code creates **time\_tuple** to represent an hour, minute, and second, displays the tuple, and then calculates the number of seconds since midnight using its elements—demonstrating that each value in the tuple is used for a different operation.

```
[11] time_tuple = (9, 16, 1)
```

```
[12] time_tuple  
(9, 16, 1)
```

```
[13] time_tuple[0] * 3600 + time_tuple[1] * 60 + time_tuple[2]  
33361
```

Assigning a value to a tuple element causes a **TypeError**.

## 3.3. Adding Items to a String or Tuple

Similar to lists, the += augmented assignment statement can be applied to strings and tuples, even though they are immutable. In the following code, after the two assignments, **tuple1** and **tuple2** will point to the same tuple object:

```
[14] tuple1 = (10, 20, 30)
```

```
[15] tuple2 = tuple1
```

```
[16] tuple2
      (10, 20, 30)
```

Concatenating the tuple **(40, 50)** to **tuple1** creates a new tuple, then assigns a reference to it to the variable **tuple1**—**tuple2** still refers to the original tuple:

```
[17] tuple1 += (40, 50)

[18] tuple1
      (10, 20, 30, 40, 50)
[19] tuple2
      (10, 20, 30)
```

For a string or tuple, the item to the right of **+=** must be a string or tuple, respectively— mixing types causes a **TypeError**.

### 3.4. Appending Tuples to Lists

You can use **+=** to append a tuple to a list:

```
[20] numbers = [1, 2, 3, 4, 5]

[21] numbers += (6, 7)

[22] numbers
      [1, 2, 3, 4, 5, 6, 7]
```

### 3.5. Tuples May Contain Mutable Objects

Let's create a **student\_tuple** with a first name, last name and list of grades:

```
[23] student_tuple = ('Amanda', 'Blue', [98, 75, 87])
```

Even though the tuple is immutable, its list element is mutable:

```
[24] student_tuple[2][1]
      student_tuple[2][1]
[25] student_tuple
      ('Amanda', 'Blue', [98, 75, 87])
```

In the expression **student\_tuple[2][1]**, Python first interprets **student\_tuple[2]** as the tuple element that holds the list **[98, 75, 87]**, and then **[1]** is used to access the element of that list which contains the grade **75**. The assignment in snippet **[24]** updates that grade to **85**.

## 4. Unpacking Sequences

The previous chapter covered tuple unpacking. You can unpack elements from any sequence by assigning the sequence to a comma-separated list of variables. If the number of variables on the left side of the assignment does not match the number of elements in the sequence on the right, a **ValueError** will be raised.

```
[1] student_tuple = ('Amanda', [98, 85, 87])
```

```
[2] first_name, grades = student_tuple
```

```
[3] first_name  
'Amanda'
```

```
[4] grades  
[98, 85, 87]
```

The following code unpacks a string, a list and a sequence produced by range:

```
[5] first, second = 'hi'
```

```
[6] print(f'{first} {second}')  
h i
```

```
[7] number1, number2, number3 = [2, 3, 5]
```

```
[8] print(f'{number1} {number2} {number3}')  
2 3 5
```

```
[9] number1, number2, number3 = range(10, 40, 10)
```

```
[10] print(f'{number1} {number2} {number3}')  
10 20 30
```

### 4.1. Swapping Values Via Packing and Unpacking

You can swap two variables' values using sequence packing and unpacking:

```
[11] number1 = 99
```

```
[12] number2 = 22
```

```
[13] number1, number2 = (number2, number1)
```

```
[14] print(f'number1 = {number1}; number2 = {number2}')  
      number1 = 22; number2 = 99
```

## 4.2. Accessing Indices and Values Safely with Built-in Function `enumerate`

Previously, we used `range` to generate a sequence of index values and then accessed list elements in a **for** loop using those indices with the subscription operator (`[]`). This approach is prone to errors because you might provide incorrect arguments to `range`, potentially leading to **IndexError** if any index is out of bounds.

A more reliable method for retrieving both an element's index and its value is the built-in **`enumerate`** function. This function takes an iterable and produces an iterator that returns a tuple with each element's index and value. The following code demonstrates how to use the list function to create a list from the results of **`enumerate`**:

```
[15] colors = ['red', 'orange', 'yellow']
```

```
[16] list(enumerate(colors))  
      [(0, 'red'), (1, 'orange'), (2, 'yellow')]
```

Similarly, the built-in function `tuple` creates a tuple from a sequence:

```
[17] tuple(enumerate(colors))  
      ((0, 'red'), (1, 'orange'), (2, 'yellow'))
```

The following for loop unpacks each tuple returned by `enumerate` into the variables `index` and `value` and displays them:

```
[18] for index, value in enumerate(colors):  
      print(f'{index}: {value}')  
      0: red  
      1: orange  
      2: yellow
```

## 5. Sequence Slicing

You can slice sequences to generate new sequences of the same type that include subsets of the original elements. Slice operations can alter mutable sequences, while those that do not change the sequence behave the same way for lists, tuples, and strings.

### 5.1. Specifying a Slice with Starting and Ending Indices

Let's create a slice consisting of the elements at indices **2** through **5** of a list:

```
[1] numbers = [2, 3, 5, 7, 11, 13, 17, 19]
```

---

|                             |                           |
|-----------------------------|---------------------------|
| [2]                         | <code>numbers[2:6]</code> |
| <code>[5, 7, 11, 13]</code> |                           |

The slice copies elements from the starting index to the left of the colon (2) up to, but not including, the ending index to the right of the colon (6). The original list is not modified.

## 5.2 Specifying a Slice with Starting and Ending Indices

If you omit the starting index, 0 is assumed. So, the slice `numbers[:6]` is equivalent to the slice `numbers[0:6]`:

|                                   |                           |
|-----------------------------------|---------------------------|
| [3]                               | <code>numbers[:6]</code>  |
| <code>[2, 3, 5, 7, 11, 13]</code> |                           |
| [4]                               | <code>numbers[0:6]</code> |
| <code>[2, 3, 5, 7, 11, 13]</code> |                           |

## 5.3 Specifying a Slice with Only a Starting Index

If you omit the ending index, Python assumes the sequence's length (8 here), so snippet [5]'s slice contains the elements of numbers at indices 6 and 7:

|                       |                                      |
|-----------------------|--------------------------------------|
| [5]                   | <code>numbers[6:]</code>             |
| <code>[17, 19]</code> |                                      |
| [6]                   | <code>numbers[6:len(numbers)]</code> |
| <code>[17, 19]</code> |                                      |

## 5.4 Specifying a Slice with No Indices

Omitting both the start and end indices copies the entire sequence:

|                                           |                         |
|-------------------------------------------|-------------------------|
| [7]                                       | <code>numbers[:]</code> |
| <code>[2, 3, 5, 7, 11, 13, 17, 19]</code> |                         |

Although slices create new objects, they perform shallow copies of the elements. This means they copy the references to the elements but not the objects themselves. As a result, in the example above, the elements of the new list point to the same objects as those in the original list, rather than to distinct copies.

## 5.5 Slicing with Steps

The following code uses a step of 2 to create a slice with every other element of numbers:

|                             |                           |
|-----------------------------|---------------------------|
| [8]                         | <code>numbers[::2]</code> |
| <code>[2, 5, 11, 17]</code> |                           |

We omitted the start and end indices, so 0 and `len(numbers)` are assumed, respectively.

## 5.6 Slicing with Negative Indices and Steps

You can use a negative step to select slices in reverse order. The following code concisely creates a new list in reverse order:

```
[9] numbers[::-1]
[19, 17, 13, 11, 7, 5, 3, 2]
```

This is equivalent to:

```
[10] numbers[-1:-9:-1]
[19, 17, 13, 11, 7, 5, 3, 2]
```

## 5.7 Modifying Lists Via Slices

You can modify a list by assigning to a slice of it—the rest of the list is unchanged. The following code replaces numbers' first three elements, leaving the rest unchanged:

```
[11] numbers[0:3] = ['two', 'three', 'five']
```

```
[12] numbers
['two', 'three', 'five', 7, 11, 13, 17, 19]
```

The following deletes only the first three elements of numbers by assigning an empty list to the three-element slice:

```
[13] numbers[0:3] = []
```

```
[14] numbers
[7, 11, 13, 17, 19]
```

The following assigns a list's elements to a slice of every other element of numbers:

```
[15] numbers = [2, 3, 5, 7, 11, 13, 17, 19]
```

```
[16] numbers[::2] = [100, 100, 100, 100]
```

```
[17] numbers
[100, 3, 100, 7, 100, 13, 100, 19]
```

Let's delete all the elements in numbers, leaving the existing list empty:

```
[18] numbers[:] = []
```

```
[19] numbers
[]
```

## 6. del Statement

The `del` statement can be used to remove elements from a list or to delete variables during an interactive session. It allows you to delete an element at any valid index or remove a range of elements from any valid slice.

### 6.1 Deleting the Element at a Specific List Index

Let's create a list, then use **del** to remove its last element:

|     |                                                                     |
|-----|---------------------------------------------------------------------|
| [1] | <code>numbers = list(range(0, 10))</code>                           |
| [2] | <code>numbers</code><br><code>[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]</code> |
| [3] | <code>del numbers[-1]</code>                                        |
| [4] | <code>numbers</code><br><code>[0, 1, 2, 3, 4, 5, 6, 7, 8]</code>    |

### 6.2 Deleting a Slice from a List

The following deletes the list's first two elements:

|     |                                                            |
|-----|------------------------------------------------------------|
| [5] | <code>del numbers[0:2]</code>                              |
| [6] | <code>numbers</code><br><code>[2, 3, 4, 5, 6, 7, 8]</code> |

The following uses a step in the slice to delete every other element from the entire list:

|     |                                                |
|-----|------------------------------------------------|
| [7] | <code>del numbers[::2]</code>                  |
| [8] | <code>numbers</code><br><code>[3, 5, 7]</code> |

### 6.3 Deleting a Slice Representing the Entire List

The following code deletes all of the list's elements:

|      |                                         |
|------|-----------------------------------------|
| [9]  | <code>del numbers[:]</code>             |
| [10] | <code>numbers</code><br><code>[]</code> |

### 6.4 Deleting a Variable from the Current Session

The **del** statement can be used to delete any variable. In this case, we'll delete `numbers` from the interactive session and then try to display its value, which will result in a `NameError`.

|      |                          |
|------|--------------------------|
| [11] | <code>del numbers</code> |
|------|--------------------------|

|      |         |
|------|---------|
| [12] | numbers |
|------|---------|

-----  
NameError

Traceback (most recent call last)

Cell In[110], line 1

----> 1 numbers

NameError: name 'numbers' is not defined

Talha Shahid